Data Query Process — Spending Pattern Walkthrough

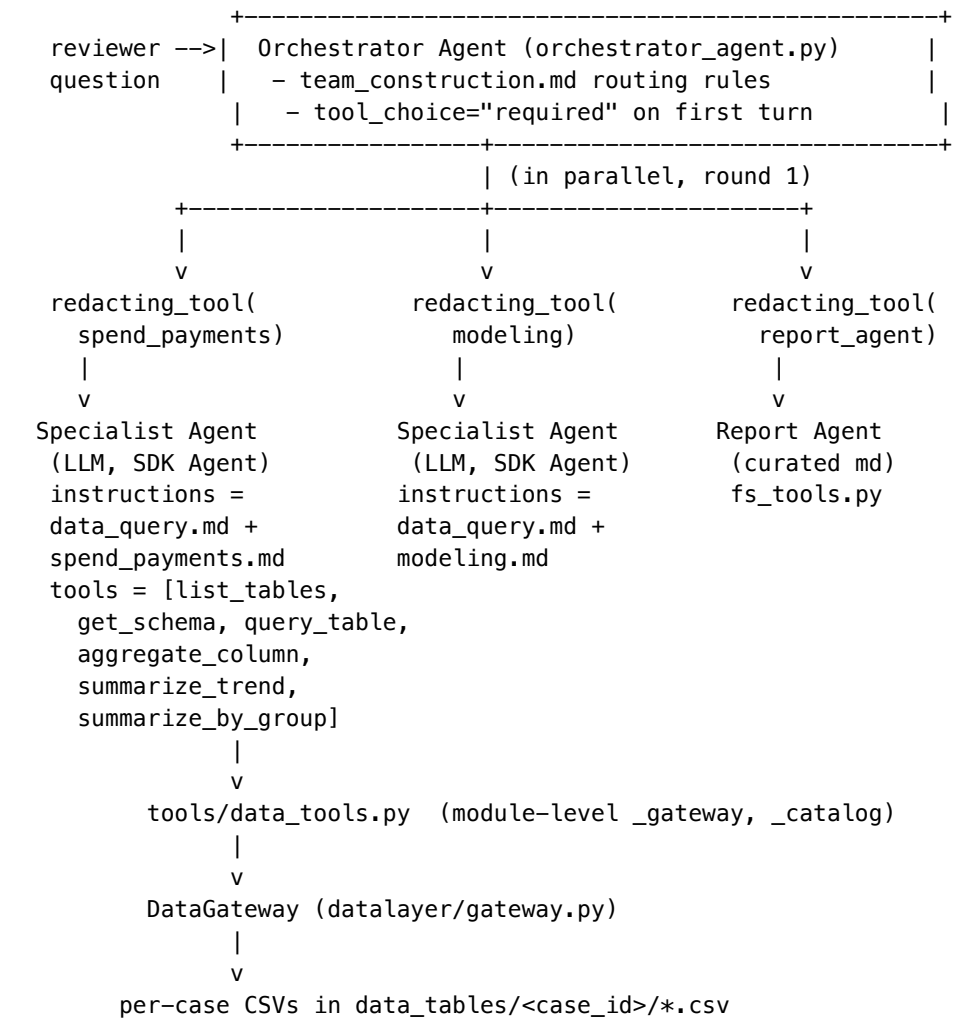
2026-05-06

Data Query Process — Spending Pattern Walkthrough

This note walks the full path a reviewer's question takes through the v2 agent graph when it asks for a spending pattern. The walk uses the exact code paths in this repo; every callout is anchored to a file and line so the wire can be traced end-to-end.

---

1. The wire at a glance



After round 1, the orchestrator (mandatory protocol from `orchestrator_agent.py:84–107`) calls `general_specialist` to compare the two specialist outputs, then emits a `FinalAnswer`.

---

2. Build-time wiring

The graph is constructed once per session by `Orchestrator.__init__` in `orchestrator/orchestrator.py:48–64`:

1. Domain skills are discovered — `list_domain_skills()` globs `skills/domain/*.md`. For "spending pattern" the relevant ones are `spend_payments.md` and `modeling.md`.
2. Specialists are built — for each domain skill, `build_specialist_agent(skill, pillar_config, model)` in `agent_factories/specialist_agent.py:56–71` creates an `agents.Agent` whose `instructions` are the concatenation of:
   - `skills/workflow/data_query.md` (the base specialist playbook, loaded once at module top, see `specialist_agent.py:20`),
   - the domain skill's `system_prompt` (the body of `skills/domain/spend_payments.md`),
   - `data_hints`, `interpretation_guide`, `risk_signals`,
   - the pillar's `concept_glossary`, `focus`, `overlay`, and `cut_off_date` block. The agent is wired with the six data tools imported from `tools/data_tools.py` and `model_settings=ModelSettings(tool_choice="requi` so the first turn must be a tool call (the model can't shortcut to a `SpecialistOutput`).
3. Report and general specialists are built — `build_report_agent(...)` (curated-`.md` reader using `fs_tools.py`) and `build_general_specialist(...)` (the cross-domain comparator that loads `skills/workflow/comparison.md`).
4. Orchestrator is assembled — `build_orchestrator_agent(...)` in `agent_factories/orchestrator_agent.py:145–186` takes the list of specialists, wraps each one as a tool via `redacting_tool(...)` (see § 3.2), then composes its own instructions from `team_construction.md`, `data_catalog.md`, `synthesis.md`, `balancing.md`, the parallel-execution / general-specialist protocol block, the pillar concept-glossary, and an auto-generated team roster (`_render_team_roster` at lines 16-59) so the LLM sees each specialist's `owns: <table>` lines verbatim.

In parallel, `tools/data_tools.py` is bound to the live data layer via `init_tools(gateway, catalog, logger)` (lines 504-516) before the orchestrator runs. After this: - `_gateway` is a `LocalDataGateway` constructed with `LocalDataGateway.from_case_folders("data_tables/")` (`datalayer/gateway.py:144–178`). The classmethod walks `data_tables/<case_id>/*.csv`, builds `{case_id: {table_name: [row_dict, ...]}}`, and runs the `_rbind_payments` post-load hook to merge `payments_success.csv` + `payments_returns.csv` into a single `payments` table with a synthetic `payment_status` column with values `"success"` / `"return"`. - `_catalog` is a `DataCatalog` reading every YAML in `config/data_profiles/*.yaml` (`datalayer/catalog.py:28–32`). Each profile carries `aliases`, `columns[*].aliases`, `dtype`, `description`, optional `categories`, etc.

The orchestrator's `Runner.run(...)` is then called with an `AppContext(gateway=..., case_folder=..., logger=...)` from `agent_factories/app_context.py`, which threads the per-specialist history map through every nested tool call.

---

3. Run-time flow for "what is the spending pattern on case X?"

3.1 Orchestrator round 1 — team selection

The orchestrator LLM reads its prompt and applies the routing table in `skills/workflow/team_construction.md:31`:

> spending / spend pattern / spend behavior / spend trajectory / spend volume / merchant concentration -> MUST include BOTH **`spend_payments`** AND **`modeling`** (+ `crossbu` only when explicitly B2B)

Combined with the TOOL-USE DISCIPLINE block in `orchestrator_agent.py:73–108`, the orchestrator is required to emit, in one parallel response:

- `spend_payments(sub_question="...")` — primary spending-pattern owner,
- `modeling(sub_question="...")` — pattern-level ML signal coverage,
- `report_agent(sub_question="...")` — pull any curated case-report context.

Every one of those three calls is wrapped by `redacting_tool` (§ 3.2).

3.2 The **`redacting_tool`** wrapper —

`agent_factories/redacting_tool.py:26–119`

Each specialist call goes through the same gauntlet:

Step	What it does	Code
1. Sanitize input	`sanitize_message(sub_question)` masks PII and `\d{6,}` runs in the orchestrator-authored sub-question before it reaches the specialist.	line 46
2. Lookup history	Read `AppContext._specialist_histories[name]`. If non-empty, prepend prior turns so a follow-up specialist call sees its own past answers.	lines 53-83
3. Dedup cache	Key `(specialist_name, normalized_subq)` against `AppContext._specialist_call_cache`. Identical sub-questions in one context return the cached payload (cuts cost when safechain mode emits near-duplicates).	lines 61-78
4. Run inner agent	`Runner.run(inner, run_input, context=app_ctx, max_turns=25)` — the 25-turn cap (`_SPECIALIST_MAX_TURNS`) gives a spending-pattern specialist room for the 11-17 tool calls its skill prescribes (§ 4.4).	lines 86-89
5. Persist history	`histories[name] = result.to_input_list()` so the next call with the same `AppContext` continues the same conversation.	lines 110-112
6. Redact output	`redact_payload(result.final_output)` masks long digit runs and `CASE-\d+` IDs on the way back up.	line 114
7. MaxTurnsExceeded	If the specialist blows the 25-turn budget, return a structured "[name] hit the budget" string instead of the SDK's generic error.	lines 90-107

3.3 Inside the **spend_payments** specialist

The Agent's `instructions` are a concatenation, top to bottom:

1. **`skills/workflow/data_query.md`** (the base). This is the common playbook for every data-querying specialist. It defines the six tools the specialist owns and their routing rules:
   - `list_available_tables()` — discovery,
   - `get_table_schema(table)` — canonical columns + aliases + `declared_values`,
   - `query_table(table, filter_column, filter_value, filter_op, columns)` — row-level fetch with auto-resolved canonical/real names and between/eq/gt/lt operators,
   - `aggregate_column(table, column, op, filter_*)` — comma-formatted `sum/mean/max/min/count`,
   - `summarize_trend(...)` — full per-period series + summary block in one call (use for any "shape over time" framing),
   - `summarize_by_group(...)` — top-N + concentration block (HHI, `top1/3/5_share`) in one call (use for any "shape across a category" framing). It also enforces the windowed-answer template, the coverage-gap disclosure, and the anti-hallucination rules (every claim in `findings` / `evidence` / `raw_data` must trace to a tool result this run produced).
2. **`skills/domain/spend_payments.md`**. The domain layer adds the spending-pattern playbook itself. The decisive section (`spend_payments.md:34–104`) lays out four dimensions a "pattern" answer must cover and the exact tool call to use for each:

§	Dimension	Tool call
A.1	volume per month	`summarize_trend('spends', 'Amount', 'Date', period='month', op='sum')`
A.2	txn count per month	`summarize_trend('spends', 'Amount', 'Date', period='month', op='count')`
B.4	recurring merchants	`summarize_by_group('spends', 'Amount', 'Merchant Name', op='count', top_n=5, sort_by='count')`
B.5	high-value merchants	`summarize_by_group('spends', 'Amount', 'Merchant Name', op='sum', top_n=5)`
B.6	per-merchant trend	`summarize_trend('spends', 'Amount', 'Date', period='month', op='sum', filter_column='Merchant Name', filter_value='<name>')` x 3-5 names
B.7	industry mix	`summarize_by_group('spends', 'Amount', 'Merchant Industry', op='sum', top_n=10)`
B.8	industry trend (optional)	`summarize_trend('spends', 'Amount', 'Date', ..., filter_column='Merchant Industry', filter_value='<industry>')`

§	Dimension	Tool call
C.9	outlier max	`aggregate_column('spends', 'Amount', op='max')` then `query_table('spends', filter_column='Amount', filter_op='gte', filter_value='<half-of-max>')`
D.11	spend ÷ successful payments	`aggregate_column('spends', 'Amount', op='sum')` and `aggregate_column('payments', 'Payment Amount', op='sum', filter_column='payment_status', filter_value='success')`
D.12	per-month spend vs. successful payments	two `summarize_trend` calls
D.13	returned-payment share	`aggregate_column('payments', 'Payment Amount', op='sum', filter_column='payment_status', filter_value='return')`

Total budget: 11-17 tool calls, well inside the 25-turn cap.

3. Hints / risk_signals / pillar glossary, which inject domain thresholds (`hhi > 0.25` = highly concentrated, `top1_share > 0.30` = single-name dominance, edge-record caveat for first/last bucket, etc.).

3.4 Concrete tool-call trace

A typical `spend_payments` run on a spending-pattern question fires the following sequence (excerpted from a real session log; actual `raw_value` numbers redacted here):

```
01  list_available_tables()                                                    # discovery
02 get_table_schema(table="spends")                                  # confirm column names + Date format
03  get_table_schema(table="payments")
04  summarize_trend("spends", "Amount", "Date", period="month", op="sum")            # A.1
05  summarize_trend("spends", "Amount", "Date", period="month", op="count")          # A.2
06  summarize_by_group("spends", "Amount", "Merchant Name", op="count",
                       top_n=5, sort_by="count")                                    # B.4
07  summarize_by_group("spends", "Amount", "Merchant Name", op="sum", top_n=5)       # B.5
08  summarize_by_group("spends", "Amount", "Merchant Industry", op="sum", top_n=10)  # B.7
09  summarize_trend("spends", "Amount", "Date", period="month", op="sum",
                  filter_column="Merchant Name", filter_value="S BERTRAM")        # B.6 #1
10  summarize_trend(..., filter_value="Dependable Plastics")                       # B.6 #2
11  summarize_trend(..., filter_value="AMEXGIFTCARD.COM")                          # B.6 #3
12  aggregate_column("spends", "Amount", op="max")                                    # C.9
13  query_table("spends", filter_column="Amount", filter_op="gte", filter_value="25000",
          columns="Date,Merchant Name,Merchant Industry,Amount")              # C.9 details
14  aggregate_column("spends", "Amount", op="sum")                                 # D.11 num
15  aggregate_column("payments", "Payment Amount", op="sum",
                filter_column="payment_status", filter_value="success")        # D.11 denom
16  summarize_trend("payments", "Payment Amount", "payment_date", period="month",
                  op="sum", filter_column="payment_status", filter_value="success") # D.12
```

```
17  aggregate_column("payments", "Payment Amount", op="sum",
                     filter_column="payment_status", filter_value="return")          # D.13
```

After turn 17, `tool_choice="required"` has long since flipped to `"auto"` (the SDK default `reset_tool_choice=True` flips it after the first tool call — see comment at `specialist_agent.py:64–71`), so the specialist now emits a `SpecialistOutput` JSON object whose `findings`, `evidence`, `implications`, and `raw_data` blocks cite the tool results by name.

3.5 Inside each tool — what actually happens

All six tools are SDK `@function_tools` in `tools/data_tools.py`. They share the exact same prologue:

```
real_table  = _resolve_real_table(table_name)         # canonical -> real CSV name
rows        = _gateway.query(real_table, filters=None) # in-memory list[dict] for active case
real_column = _resolve_real_column(rows, column, real_table)  # canonical -> real CSV header
rows        = _apply_filter(rows, real_column, filter_value, filter_op)
```

The two resolvers are the glue between the LLM's vocabulary (`"spends"`, `"Amount"`, `"Date"`) and the real CSV headers (which may be `spends_data.csv`, `Transaction Amount`, etc.):

- `_resolve_real_table` (lines 365-418) cascades: exact match -> catalog table-level alias -> `<requested>_data` convention -> reverse strip `_data` -> normalized fuzzy.
- `_resolve_real_column` (lines 421-459) cascades: exact match in row keys -> catalog-declared column alias -> normalized fuzzy.

The actual aggregation:

- `_apply_filter` (lines 462-501) handles `eq/ne/gt/gte/lt/lte/between`. Range ops use `_coerce_pair` to coerce to numeric, then to a `(year, month, day)` tuple via `_date_key` (handles ISO, `October'2024`, `Oct–2024`, `07–Jul–2024`, etc.), then string. So a `between` on a `payment_date` column compares chronologically across mixed formats.
- `_format_aggregate` (lines 972-987) is the redaction-survival trick — every numeric return value gets thousand separators, so a `$174,897.36` survives the boundary `\d{6,}` mask that would turn raw `174897.36` into `***MASKED***.36`. This is the whole reason `aggregate_column` exists rather than letting the LLM sum `query_table` rows itself.
- `summarize_trend` (lines 1310-1548) buckets via `_bucket_key`, computes `slope_per_bucket` with `_slope` (OLS), `coefficient_of_variation`, `pct_change_first_to_last`, and a **`missing_periods`** list via `_enumerate_periods` so a gap in the data shows up as a labeled finding ("missing 2025-04, 2025-05") instead of being silently dropped.
- `summarize_by_group` (lines 1627-1848) groups by a categorical column and computes the **`concentration`** block (`hhi`, `top1/3/5_share`) when the op is additive (`sum` or `count`).

Each tool also calls `_log_call` and `_log_result` (lines 528-552), so the session's `EventLogger` gets a `tool_call` event (with args) and a `tool_result` event (with row count + 500-char preview) for every hop. That log is what makes the trace in § 3.4 reproducible.

3.6 Round 2 — **`general_specialist`**

After round 1's three results land back in the orchestrator's context, the mandatory two-round protocol in `orchestrator_agent.py:84–107` forces the orchestrator to emit a single `general_specialist(...)` call. Its skill (`skills/workflow/comparison.md`, loaded by `agent_factories/general_specialist.py:15`) compares the `spend_payments` and `modeling` outputs, surfaces contradictions, and returns a `ReviewReport`. Only after that does the orchestrator emit a `FinalAnswer`.

`general_specialist` has no tools (`tools=[]` on `general_specialist.py:22`) — it's pure synthesis, fed exclusively by the in-context outputs from round 1.

3.7 Final synthesis

The orchestrator's `output_type=AgentOutputSchema(FinalAnswer, strict_json_schema=False)` forces it to emit a `FinalAnswer` Pydantic object. `Orchestrator.run` (`orchestrator.py:78–88`) wraps the result in one more `redact_payload` on the way out, logs `orchestrator_run_done`, and returns the final to the caller.

---

4. Tool-by-tool reference (with the spending-pattern uses)

Tool	Where	Args	Spending-pattern role
`list_available_tables`	`data_tools.py:599`	`()`	Confirms `spends`, `payments`, `txn_monthly` are present for this case.
`get_table_schema`	`data_tools.py:714`	`table_name`	Gets real column names + canonical/alias map. Must run before filtering on a column whose vocab the specialist hasn't seen — the skill's "Schema & vocabulary" rule.
`query_table`	`data_tools.py:918`	`table_name, filter_column, filter_value, filter_op, columns`	Used in C.9 to fetch the rows above 0.5 x max amount with their `Date`, `Merchant Name`, `Merchant Industry`. Returns `{table, filter, total_rows_in_table, rows_matching_filter, rows_returned, truncated, rows[...]}` — counts always come from `rows_matching_filter`, never from `len(rows)`.
`aggregate_column`	`data_tools.py:1151`	`table_name, column, op, filter_*`	D.11/D.13 totals + C.9 max. Returns a single comma-formatted line that survives redaction. Date-aware fallback for `max/min` on string-date columns.
`summarize_trend`	`data_tools.py:1552`	`table_name, value_column, time_column, period, op, filter_*, start_date, end_date`	A.1, A.2, B.6 x 3-5, D.12. One call == a whole bucketed series + first/last/peak/trough/total/mean/slope/C The skill explicitly forbids looping `aggregate_column` per period.

Tool	Where	Args	Spending-pattern role
summarize_by_group	data_tools.py:1852	table_name, value_column, group_column, op, top_n, sort_by, filter_*	B.4, B.5, B.7. One call == top-N groups + concentration{top1_share, top3_share, top5_share, hhi} for additive ops.

---

5. Cross-cutting concerns

- Module-level state, autoreload-safe. tools/data_tools.py:23–34 guards _gateway, _catalog, _logger with try/except NameError so a notebook's %autoreload 2 doesn't silently null them.
- Boundary redaction. Three layers, all using the same \d{6,} pattern: llm.firewall_stack.redact_payload (output of every specialist call), llm.firewall_stack.sanitize_message (input of every specialist call), and the format-on-read in tools/fs_tools.py:24–48 for curated .md files. The numeric formatters in _format_aggregate and the canonical _LONG_NUMERIC_RE regex in fs_tools.py are the dual sides of the same defensive layer.
- Per-specialist memory. AppContext._specialist_histories is a dict, keyed by tool name. The wrapper writes it after every run and reads it on every entry, so a follow-up "and how did Industrial Supplies trend month-by-month?" reuses the same spend_payments agent and continues its own thread instead of re-doing the discovery calls.
- Per-specialist dedup. AppContext._specialist_call_cache (attached lazily on first use, lines 62-69 of redacting_tool.py) caches identical normalized sub-questions. Important under safechain mode where parallel-tool-call semantics aren't native and the orchestrator may emit the same call twice.
- Forced first turn. Both specialists and the orchestrator are built with ModelSettings(tool_choice="required"). reset_tool_choice (SDK default True) flips back to "auto" after the first tool call, so the model is forced to ground its first move in real data but is free to synthesize the structured output afterwards. Without this, some models hallucinate "I was unable to access the schema" instead of calling get_table_schema.
- Turn budget. _SPECIALIST_MAX_TURNS = 25 in redacting_tool.py:14. The 11-17 tool-call spending-pattern budget in spend_payments.md:102 is sized to this cap with margin.
- Pillar cut-off date is injected into every specialist's prompt by specialist_agent.py:40–52. All "recent / last N months / this year" language is anchored to cut_off_date, never today. The base data_query.md also enforces a coverage-gap disclosure when the asked window exceeds the actual observed range.

---

6. TL;DR

A spending-pattern question hits three SDK Agents (spend_payments, modeling, report_agent) in parallel, plus a fourth (general_specialist) for cross-domain comparison. Inside spend_payments, data_query.md + spend_payments.md together prescribe 11-17 calls to the six function_tools in tools/data_tools.py, which read the in-memory case CSVs through LocalDataGateway with canonical-name resolution from DataCatalog. Every numeric tool result is comma-formatted to survive boundary redaction, every specialist input/output is sanitized at the redacting_tool boundary, and per-specialist conversation history + dedup live on AppContext so follow-ups continue the same specialist thread cheaply.